



Jose Parreño Garcia <joparga3@gmail.com>

[TEST r0t - Paid] What the docs don't tell you about Claude Code skills

1 mensaje

Jose Parreño Garcia from Senior Data Science Lead <joseparreogarcia@substack.com>
Para: joparga3@gmail.com

30 de marzo de 2026 a las 10:52

[View in browser](#)

The Senior Data Science Lead newsletter

Hi! It's Jose from Senior Data Science Lead newsletter, where I write about senior data science leadership, sharing hard-earned lessons from building teams, systems, and real-world machine learning.

Because you are a paid subscriber, you are directly supporting this work — and you get full access to the practical tools built alongside the writing:

- ***The full Data Storytelling course, focused on turning analysis into insights that land with executives. Because today, anyone can build a chart with AI, but very few people can build the right chart.***



Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

- ***50+ Notion templates for hiring, onboarding, mentoring, and planning built for Data and Tech Leads. As a paid subscriber, you get **50% off** with code **PAIDSUBS-03PL**.***



50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)

And as always, thank you for supporting the newsletter — your support is what keeps it independent, practical, and focused on real-world impact.

What the docs don't tell you about Claude Code skills

A practical guide to the anatomy, hidden levers, and design principles that make Claude Code skills reliable.

JOSE PARREÑO GARCIA

MAR 30 · PAID · DRAFT





READ IN APP ↗

This month, I finished building a skill for every stage of my newsletter writing pipeline.

The process was straightforward, because everyone just told me: *“hey, its easy, a skill is just a well defined markdown file”*.

Now, Anthropic docs tell you how to create a skill. They also cover some good practices, but those are scattered across the official spec, production examples. Most importantly, it’s only after a fair amount of direct observation where you start to understand if the skill is doing its job or not.

So, what I wanted to share with you is that **skills are a design pattern, not just a feature**. Most practitioners treat them as markdown files with instructions. That is half right (great for PoCs) but you can enhance them much more for production grade systems, where reliability is key..

This post assumes you have already encountered Claude Code skills and want to go deeper. I will not walk you through creating your first skill. I will cover what the docs underexplain, and what I have found that actually makes skills reliable.

What this blog will cover

- **Claude skills in under 1 minute.** A quick check on what they are and what problem they solve.
- **The mental model most practitioners are missing.** Why a skill is not just instructions, and what actually happens under the hood.
- **The full anatomy of a skill.** SKILL.md, frontmatter, and the directories that make the system work.
- **The hidden levers behind reliable skills.** The small design decisions that determine whether a skill works consistently or fails silently.
- **Advanced tricks that unlock real power.** Mixing scripts with reasoning, designing for composability, and controlling model behaviour.
- **How skills fit into the wider system.** Where they sit relative to MCP, subagents, and agentic workflows.
- **When skills are the wrong tool.** The boundaries that keep your system simple and maintainable.
- **Practical design principles.** How to structure, test, and iterate on skills as if they were software.

Let's get started!

Claude skills in under 1 minute: what are skills?

A Claude skill is a reusable package of instructions, resources, and optional code that teaches Claude how to **perform a specific task *reliably*** (*well, reliably depends really on how you define this skill, and that's what we will cover in this post*).

Unlike a prompt, which lives in a single conversation, a skill persists across sessions and can be selected automatically when relevant. It is not just something Claude reads, it is something Claude can *decide to use*.

At a glance, a skill looks simple: a markdown file with some instructions. And for small use cases, that is often enough. Simple right?

The mental model most people miss

Well, of course it is simple if you scratch the surface of skills. As mentioned above, most practitioners mentally model a skill as: a markdown file, instructions at the top, maybe some examples. Claude reads it, follows it, done. Something like this:

```
---  
name: summarise-meeting-notes  
description: Summarises raw meeting notes into a short structured  
recap. Use when the user provides messy notes and wants clear  
next steps.  
---  
  
## Instructions
```

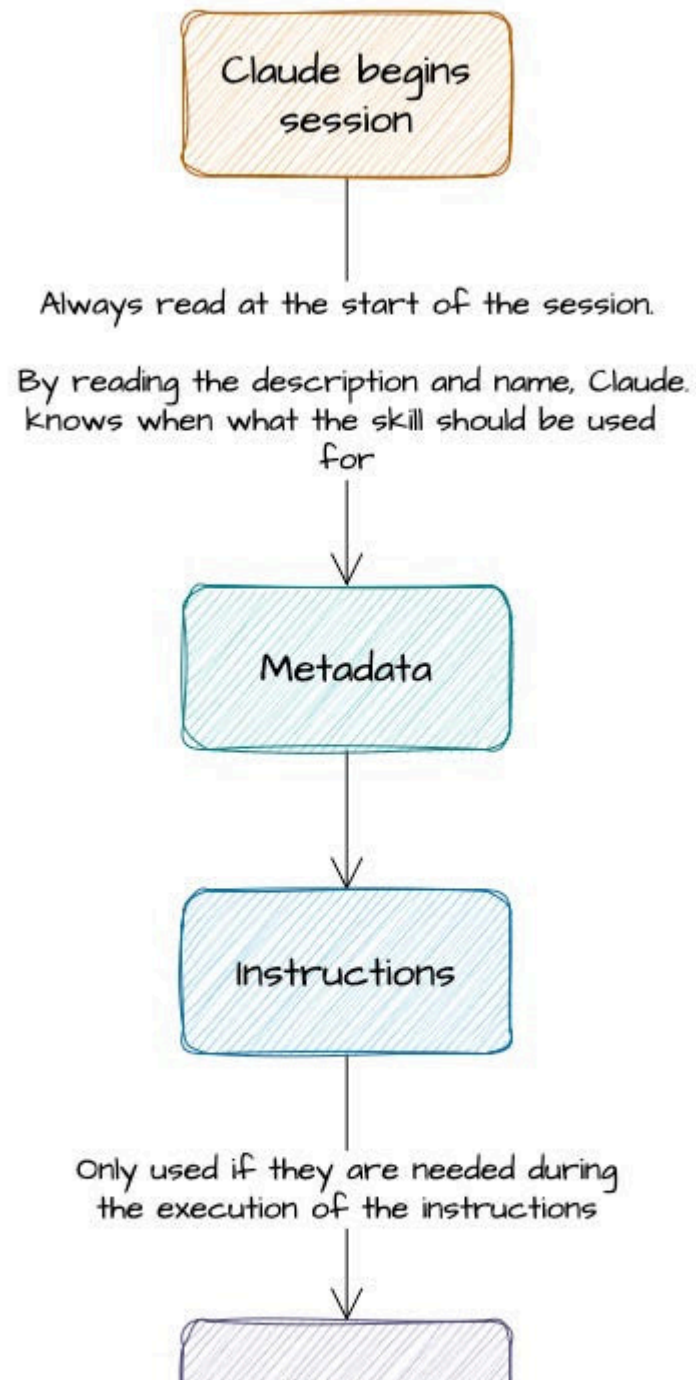
- Read the notes carefully.
- Extract the main decisions, open questions, and action items.
- Return the output under these headings:
 - Summary
 - Decisions
 - Action items

That is the version most people carry around in their heads: *a skill is basically a markdown prompt with a name attached*. Now, this of course works. And it great for PoCs.

But that mental model breaks down quickly as soon as you try to make skills reliable at scale. This is why skills are better understood as a design pattern, not just a feature. The [Agent Skills Overview in the Claude API docs](#) however, describes something more precise: a 3-layer architecture where different parts of a skill load at different times, at different costs.

- **Metadata (a.k.a. *Frontmatter fields*)** — always loaded at startup. Claude reads this first, for every available skill, in order to decide what might be relevant. This is why the description is **SO** important for skills (more on that later).
- **Instructions** — This is the main body of `SKILL.md`. It gets loaded when the skill is actually selected. This is where the workflow, steps, examples, and output expectations live.
- **Resources** — These are supporting files such as documents, templates, schemas, or scripts placed in directories like `references/`,

assets/, or scripts/. They are loaded only *when needed* during execution.



References

These are assets that, if you understand well, they can make skills take a massive leap of quality for your production systems. This is also why skills are better understood as a **design pattern**, not just a feature. Now that we have the mental model, we can look at the actual anatomy of a skill and see where each of those layers lives in practice.

The full anatomy of a skill

Now that the loading model is clear, we can look at the physical anatomy of a skill: what files it contains, what each part is responsible for, and how that structure supports the layered design we just covered. These are the elements we will cover:

1. The skills directory and the SKILL.md
2. The 3 optional folders you can use in a skill.
3. A deep dive into the fields that you can use in the Frontmatter fields

The skills directory and the SKILL.md

At the simplest level, a skill is just a directory with a required SKILL.md file inside it.

In Claude Code, skills typically live under the `.claude/skills/` directory. Each skill gets its own folder, and the folder name becomes the skill's identity on disk. Inside that folder, `SKILL.md` is the one file that must exist.

A minimal skill might look like this:

```
.claude/  
└─ skills/  
    └─ summarise-meeting-notes/  
        └─ SKILL.md. # Word of caution -> this has to be named  
            EXACTLY SKILL.md
```

That alone is enough to create a working skill. But in practice, robust skills usually grow a bit more structure around that core.

The 3 optional folders you can use in a skill.

3 optional directories sit alongside `SKILL.md`:

- `scripts/` — executable code. Shell scripts, Python files, anything the skill can invoke directly. This is where deterministic execution lives.
- `references/` — heavy context loaded selectively. Design specifications, lookup tables, large reference documents. These load on demand, not at activation.
- `assets/` — supporting files: templates, schemas, example outputs.

This means that a more complete skill might look like this:

```
.claude/  
└─ skills/  
    └─ newsletter-draft-review/  
        ├── SKILL.md  
        ├── scripts/  
        │   └─ validate_structure.py  
        ├── references/  
        │   └─ style_guide.md  
        └─ assets/  
            └─ output_template.md
```

The `SKILL.md` file holds the metadata and main instructions. But the optional folders let you keep the skill lean by moving bulky or specialised material out of the main file and loading it only when needed. That matters because content placed in `references/` does not have to be loaded every time the skill activates, whereas content stuffed into the body of `SKILL.md` does.

It's worth paying a bit more attention to the `references/` directory. It is where the three-layer architecture pays off in practice. Heavy content placed there loads only when the skill needs it during execution. The same content placed in the `SKILL.md` body loads at every activation. That distinction has a real token cost, and across a system of many skills it compounds.

The practical guideline from the [Anthropic best practices documentation](#): keep `SKILL.md` under 500 lines. Treat that as a structural target. When a skill

body exceeds it, the question is not “how do I make the prose tighter” — it is “what belongs in references/?”

A deep dive into the fields that you can use in the Frontmatter fields

Inside `SKILL.md` itself, the top section is the frontmatter: a small YAML block that describes the skill. The Frontmatter fields are nothing more than specs which describe the skill.

Now, you can't add anything you want in the specs, this section needs an actual format or schema. As of time of writing, these are the fields you can work with:

1. *(required)* `name`. Lowercase, hyphenated, and concise.
2. *(required)* `description`. Explains what the skill does and when it should be used.
3. *(optional)* `license`. A licence name or reference to a bundled licence file.
4. *(optional)* `compatibility`. Environment requirements such as product target, system packages, or network assumptions.
5. *(optional)* `metadata`. Extra key-value information such as owner, version, or internal tags.
6. *(experimental)* `allowed-tools`. A space-delimited list of tools the skill may use.

7. *(experimental)* argument-hint. Supported in Claude Code skills as a usability field for slash-style invocation, but it is not part of the core open Agent Skills specification.

Here is a possible example.

```
---
name: newsletter-draft-review
description: Reviews a newsletter draft for structure, clarity,
and section flow. Use when the primary input is a draft article
and the goal is editorial improvement rather than full rewriting.
license: Apache-2.0
compatibility: "Claude Code; requires Python 3.11+ for local
validation script"
metadata:
  author: jose-parreno-garcia
  version: "1.2"
  team: senior-data-science-lead
allowed-tools: Read Write Bash(python:*)
---
```

An example of how Anthropic uses skills.

As you can see, skills can be way more powerful than 1 simple script with markdown instructions. In fact, these same features are what Anthropic themselves use.

In [their skills repository](#), Anthropic has an incredible amount of examples that we can all learn from. Here are 2 to show you:

1. [xlsx skill](#). 292 lines and worth reading in full. The description field is long and detailed — explicit triggers ("DO trigger when...") and explicit exclusions ("Do NOT trigger when..."). The body sets strict output standards: zero formula errors, professional formatting conventions throughout.
2. [frontend-design skill](#). Full use of everything a skill can use. It has a scripts/ and references/ directories. On top of that agents/ and many other directories. A really really complete skill.

These are not verbosity. It is precision engineering. So yes, learn from them.

The hidden levers that separate reliable skills from fragile ones.

Once you understand the anatomy of a skill, the next question is more interesting: **which parts actually determine whether the skill behaves reliably in practice?**

The description field selects the skill

The first, and most underestimated, is the description field.

It is worth repeating that, Claude does **not** begin by reading every SKILL.md in full. It starts with the metadata layer, especially the name and description,

and uses that to decide which skills might be relevant. Only after a skill is selected does the main body get loaded. When Claude has 100+ skills available this can be a deal breaker in terms of context bloat.

So, let me repeat it yet again:

The description is not just documentation, it is the selection interface.

The [best practices guide](#) is direct about this: write descriptions in third person, include both what the skill does and when to use it, include specific trigger keywords. Description quality becomes a ranking signal as the number of available skills grows.

What that implies in practice: **the most important content in a skill file is often the content practitioners spend the least time on.**

The xlsx skill's description explains exactly which spreadsheet operations should trigger the skill, which should not, and what the skill guarantees about output quality.

name	xlsx
description	Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xslm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from scratch or from other data sources; or convert between tabular file formats. Trigger especially when the user references a spreadsheet file by name or path — even casually (like "the xlsx in my downloads") — and wants something done to it or produced from it. Also trigger for cleaning or restructuring messy tabular data files (malformed rows, misplaced headers, junk data) into proper spreadsheets. The deliverable must be a spreadsheet file. Do NOT trigger when the primary deliverable is a Word document, HTML report, standalone Python script, database pipeline, or Google Sheets API integration, even if tabular data is involved.
license	Proprietary. LICENSE.txt has complete terms

Screenshot for Anthropic’s skill description.

Encoding failure modes at the selection layer

A skill that handles only the happy path is a skill waiting to fail.

Explicit exclusions in the description (“*Do NOT trigger when...*”) prevent the skill from being invoked in contexts where it will produce bad output. The failure mode is encoded at the selection layer, before any instructions run. This is categorically different from handling failure modes in the body — by the time the body runs, the wrong skill has already been selected.

If you read again the screenshot of Anthropic’s skill description, it clearly states: “*Do NOT trigger when the primary deliverable is a Word document ...*” This line should prevent the wrong skill from being selected in the first place. Takeaway: if Anthropic use it, so should you.

Structuring outputs for downstream use

A skill that ends in free-form prose may still be useful to a human reader. But it is much harder to compose into a larger workflow than a skill that returns something structured and predictable.

If another skill, script, or review step needs to consume the output, then output shape becomes part of the interface. That means the skill should not just “answer well”. It should answer in a way that another step can reliably use.

That structure does not have to mean rigid JSON every time. It can be:

- YAML,
- JSON,
- or simply fixed markdown headings with predictable sections.

The important part is that the structure is deliberate and stable. The [Claude Skills Cookbook](#) shows this pattern clearly in multi-step workflows, where one stage produces output designed to feed the next cleanly.

This is where composability begins.

Advanced tricks: scripts, composability, and model routing.

Once the basics are in place, the next step is to make the skills design **more deliberate**.

Anthropic’s documentation and the open Agent Skills material both treat skills as more than instruction files: they are folders of instructions, scripts, and resources designed to work together. Here are the ones that I try to really ensure my skill design cover.

Scripts that mix reasoning and determinism.

One of the most useful things a skill can do is split work between **Claude's judgement** and **plain executable code**.

This is what the `scripts/` directory is for. In both Claude Code and the broader Agent Skills model, skills can include executable scripts alongside instructions and reference files. That means the skill does not have to rely on generated text for everything. It can ask Claude to reason about what needs doing, then hand a precise subtask to Python, Bash, or another script when the job is better handled deterministically.

That division of labour is more important than it sounds. Claude is useful for interpretation, prioritisation, classification, and decision-making. Scripts are better for repeatable mechanics: validating a schema, checking section structure, normalising a file, extracting fields, or enforcing a formatting rule. In other words, **let Claude decide, let code verify**. That is usually a stronger design than asking the model to improvise both

Separation of concerns (or modularity)

The second advanced trick is composability.

A narrow skill often looks weaker than a large one when viewed in isolation. But in practice, narrow skills are usually easier to select, easier to maintain, and easier to combine. Anthropic's guidance repeatedly pushes in this direction: skills should be concise, focused, and structured so Claude can discover and use them successfully. Claude Code also places skills alongside

other modular components such as hooks, agents, and MCP servers rather than treating them as giant self-contained applications.

The **design principle is composability over monoliths**: a system of focused skills that chain cleanly outperforms one skill trying to handle everything.

Model selection (but worth framing carefully)

In Claude Code today, there appears to be a behaviour in SKILLS, where we can force the skill to use it (despite you working in your session with another model).

 *The important caveat is that I would not present per-skill model selection as part of the stable, documented cross-platform skill specification.*

Here is an example of how you would use it:

```
---  
name: newsletter-draft-review  
description: Reviews a newsletter draft for structure, clarity,  
and section flow. Use when the primary input is a draft article  
and the goal is editorial improvement rather than full rewriting.  
license: Apache-2.0  
compatibility: "Claude Code; requires Python 3.11+ for local  
validation script"  
metadata:  
  author: jose-parreno-garcia  
  version: "1.2"
```

```
team: senior-data-science-lead
allowed-tools: Read Write Bash(python:*)
model: Sonnet
---
```

And here is an example how, is one of my skills, this behaviour would trigger. In the screenshot below, I had Opus as my main session model, but the skill called `fetch-calibration-data` used Haiku.

```
↑ Opus now defaults to 1M context · 5x more room, same pricing
> /model
  ↳ Set model to Opus 4.6 (1M context)
> use the fetch-calibrator data skill for the managers team.
  ↳ @ Selected 7 lines from .claude/skills/fetch-calibration-data/SKILL.md in Visual Studio Code
• Skill(fetch-calibration-data)
  ↳ Successfully loaded skill · 3 tools allowed · claude-haiku-4-5-20251001
Queried atlassian (ctrl+o to expand)
  ↳ Interrupted · What should Claude do instead?
> 
? for shortcuts
```

using Opus in my main session...

however, because I had Haiku defined, the skill used Haiku!

Up until now, we have seen some great tricks for skills. However, skills become much more powerful once they start interacting with the wider Claude system — especially MCP for external access, and subagents for delegated execution. Let's cover that briefly in the next section.

How skills fit with MCP, subagents, and agentic pipelines.

A skill is a **workflow layer**. It teaches Claude what to do, in what order, and to what standard. But a real system usually needs more than that. It may also need access to external data, or a separate execution context for specialised work. That is where MCP and subagents enter the picture.

The cleanest way to think about the 3 is this:

- **MCP provides access**
- **skills provide workflow logic**
- **subagents provide execution context**

The [skills explained blog post](#) from Anthropic includes a comparison table across five primitives: Skills, Prompts, Projects, Subagents, and MCP. The table covers persistence, what each contains, when it loads, and best-for scenarios. The two integrations that matter most in practice are skills + MCP and skills + subagents.

Feature	Skills	Prompts	Projects
What it provides	Procedural knowledge	Moment-to-moment instructions	Background knowledge
Persistence	Across conversations	Single conversation	Within project
Contains	Instructions + code + assets	Natural language	Documents + context
When it loads	Dynamically, as needed	Each turn	Always in project
Can include code	Yes	No	No
Best for	Specialized expertise	Quick requests	Centralized context

Screenshot from [Anthropics blog post](#).

Skills + MCP

The [December 2025 post on extending Claude's capabilities](#) uses an analogy I find precise: MCP servers are aisles of inventory, skills are employee expertise. MCP gives Claude access to data and tools. Skills give Claude a defined process for using them.

Neither alone is sufficient for real agentic workflows. A skill without MCP has to work with whatever is already in context. MCP without a skill gives Claude data access and no structured process for acting on it. Together, they close the loop: access the right data, apply the right process.

For example, an MCP server might expose a Notion workspace and document how to search it correctly. A skill would then define the actual workflow: check the project page first, then previous meeting notes, then stakeholder context, and finally return the summary in a fixed structure.

Skills + subagents

The relationship with subagents is different.

- Skills define what to do and in what sequence.
- Subagents provide execution capacity and domain focus.

This difference is important because these two primitives are sometimes conflated. Skills define the *how*, agents define *who is doing the work* (and do it).

Skills in agentic pipelines

Once you combine these layers, skills stop being isolated utilities and start looking like building blocks in a larger pipeline.

- One step might use a skill to define a workflow.
- Another might use MCP to fetch live data.
- Another might run inside a subagent so the work happens in an isolated context.
- And the final output might feed into another skill or a human review stage.

That is what makes the broader architecture feel agentic.

This is also why trying to force everything into a single skill usually goes wrong. As soon as a workflow needs both external access and controlled execution, the better design is usually not “make the skill smarter”. It is **“give the skill the right neighbours”**.

When skills are the wrong tool.

Everything we have covered should already provide us with a view of where skills are more useful (and hence, when they are not). Still, here are 4 bullets that I mentally keep so that I don't go off building skills like a mad man.

- **One-off tasks.** If you are only going to do something once, a prompt is usually enough. Skills carry authoring overhead. You need to name them, describe them, structure them, test them, and maintain them. That investment only pays off when the workflow repeats often enough for

reuse and iteration to matter. This is an easy trap to fall into, especially when building with Claude feels fast. A task works once, so it is tempting to immortalise it immediately as a skill.

- **Static context delivery.** If the “skill” is a block of reference text with no workflow logic, it belongs in a [CLAUDE.md](#) rules file or other rules layers. Skills are meant to encode **how to do a task**. Static context is about what Claude should know while doing many tasks.
- **External data access without MCP.** Skills that reach directly into external APIs or databases are fragile — difficult to test, difficult to maintain, and blurring a boundary that exists for a reason. Data access is MCP’s job. What to do with the data is the skill’s job. Conflating them produces skills that are hard to debug when the data source changes.
- **The “one skill to rule them all” anti-pattern.** A monolithic skill that handles research, drafting, SEO optimisation, and promotion in one file is not comprehensive — it is un-testable, un-discoverable, and un-maintainable. When a skill tries to handle multiple unrelated workflows, it selects inconsistently and fails silently in the cases it was not designed for.

Practical design principles, including how to test skills like code.

The [best practices guide](#) and the [evaluation guide from agentskills.io](#) together provide the clearest operational guidance available. These are the principles I apply consistently.

- **Narrow and discoverable.** A skill that does one thing reliably is more useful than one that handles many things inconsistently. Narrow scope makes skills easier to select, easier to test, and easier to reason about when they fail.
- **Descriptions written for selection.** Write in third person. Include both what the skill does and when to use it. Use specific keywords that reflect how users will naturally describe the task. Treat the 1,024-character limit as design space — not a ceiling to stay comfortably below, but a budget to deploy thoughtfully. The description is the only part of the skill Claude reads before deciding whether to activate it.
- **Separate instructions from heavy context.** [SKILL.md](#) carries instructions. [references/](#) carries context. Anything that is reference material rather than workflow logic — style guides, specification documents, lookup tables — belongs in [references/](#). Keeping [SKILL.md](#) lean preserves the three-layer loading model's efficiency.
- **Test like code.** The [agentskills.io evaluation framework](#) treats skills as software artefacts: design test cases (prompt + expected output + any input files), run with-skill and without-skill comparisons, write assertions about the output, grade and iterate. This is a complicated topic on its own that grants some future posts. For the moment, its enough to know it is important to consider too.

Closing thoughts

Writing a prompt is a conversational act. Building a skill is a *design* act.

You have to:

- Define scope
- Encode failure modes
- Separate instructions from context
- Structure outputs for downstream use
- And test the result.

I see these as software engineering design skills applied to a new medium.

The [original Agent Skills announcement](#) frames skills as “composable, portable, efficient.” The open standard at [agentskills.io](#) signals where this is heading: skills designed to spec travel across Claude clients and, as the standard matures, across other agent implementations. A skill you build carefully today in Claude Code could run in a different agent context tomorrow.

Now, I want to hear from you

If you have been building Claude Code skills:

- What is the most non-obvious design decision you have made — and did it hold up?
- Have you hit the “one skill to rule them all” trap? How did you climb back out?

- Are you testing skills systematically, or still finding problems in production?

I am particularly curious about how people are combining skills with MCP in practice. The integration patterns feel like they are still being worked out in the wild — I would like to compare notes.

Further reading

If you are interested in more content, here is an article capturing all my written blogs!



All my articles, one convenient place

JOSE PARREÑO GARCIA · OCTOBER 15, 2024

[Read full story](#)

References

[1] [Introducing Agent Skills](#) — Anthropic's original October 2025 launch announcement for Agent Skills; covers composable/portable/efficient framing and the December 2025 note on [agentskills.io](#) as an open standard.

[2] [Agent Skills Overview — Claude API Docs](#) — The most technically precise public source on the three-layer loading architecture (metadata, instructions, resources) with token estimates; also covers the virtual machine execution model.

[3] [Agent Skills Specification — agentskills.io](#) — The canonical open-standard spec for [SKILL.md](#) format; covers all frontmatter fields including the

experimental `allowed-tools` field and the `scripts/`, `references/`, `assets/` directory structure.

[4] [xlsx SKILL.md — anthropics/skills](#) — A 292-line production Anthropic skill demonstrating real description field design with explicit triggers and exclusions.

[5] [Skill Authoring Best Practices — Claude API Docs](#) — Covers description writing conventions, naming standards, and the 500-line [SKILL.md](#) guideline; includes the insight that description is the selection ranking signal among 100+ skills.

[6] [Claude Skills Cookbook README — anthropics/claude-cookbooks](#) — Practical builder's guide with financial workflow examples demonstrating progressive disclosure and structured output patterns.

[7] [Using Scripts in Skills — agentskills.io](#) — Definitive guide to scripting inside skills; covers the deterministic vs generative distinction and design principles for scripts in agentic contexts.

[8] [Skills explained: How Skills compares to prompts, Projects, MCP, and subagents](#) — Comparison table across five Claude primitives; the clearest source on when skills are not the right tool.

[9] [Extending Claude's capabilities with skills and MCP servers](#) — The canonical source on the skills + MCP integration; uses the hardware store analogy and real-world workflow examples. Published December 2025.

[10] [Evaluating Skill Output Quality — agentskills.io](#) — Covers the eval loop for skills as software artefacts: test case design, with-skill vs without-skill comparisons, assertions, and iteration workspace structure.

Thanks again for supporting the Senior Data Science Lead newsletter.

Your support makes it possible for me to keep writing openly, while also investing time in the more structured, hands-on work — courses, templates, and tools — that help people apply these ideas in practice. You can find them [here](#).



 50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)



 Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

This newsletter exists in its current form because of readers like you. I am genuinely grateful for that.

Thank you.

 LIKE

 COMMENT

 RESTACK

Gmail - [TEST r0t - Paid] What the docs don't tell you about Claude Code skills
© 2026 Jose Parreño Garcia
548 Market Street PMB 72296, San Francisco, CA 94104
[Unsubscribe](#)

Get the app

 Start writing